BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

LESSON 111 · Insertion Sort

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. When does ascending bubble sort swap two adjacent items?

Q2. What does readLine() do on an open text file?
A. Writes a new line
B. Deletes the next line
C. Reads the next line
D. Closes the file

Q3. “Primary storage exists only to keep long-term backups.” Correct this. Explain why.

Q4. “My unsaved work will survive a power cut because this computer has ROM.” Explain the mistake. How could the work have been kept?

BLACK PEN Answer all questions first. GREEN PEN Then: tick and rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

Insertion sort — Builds a growing sorted prefix by inserting each next value.

Key — Current element being inserted.

Sorted prefix — Left section already in order.

Shift — Move an existing value one place right to make room.

New learning

Build a sorted prefix: Treat the first element as a sorted section, then take the next element as a key.

Make space: Move larger sorted elements one position right to create room for the key.

Insert the key: Place the key in the correct position in the existing sorted prefix.

Grow the prefix: Repeat with each remaining element until the whole list is ordered.

Worked example

Trace insertion sort on [7,3,5,1,4], showing the sorted prefix after each insertion. [5 marks]

1. Insert 3

2. Move 7 right; result [3,7,5,1,4].

3. Insert 5 → [3,5,7,1,4]; insert 1 → [1,3,5,7,4].

4. Insert 4 → [1,3,4,5,7]. All five values are sorted. [5]

We Do · Practise with guidance

Guided 1

Trace the first two insertions of [6,2,4,1]. [4]

Worked method: After inserting 2: [2,6,4,1] [2]; after inserting 4: [2,4,6,1] [2].

Guided 2

Insert key 3 into sorted prefix [1,4,6] (next value 3). [3]

Worked method: Shift 6 right [1], shift 4 right [1]; insert 3 after 1 [1], prefix [1,3,4,6].

Guided 3

Explain why insertion sort can be efficient on nearly sorted data. [3]

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Worked method: Many keys are near their correct position [1], so fewer shifts/comparisons are needed [1] than when elements are far out of order [1].

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Code First · Predict → Write → Test → Improve

Complete the line that shifts the larger value during insertion.

```
array depths = [16,8]
key = depths[1]
if depths[0] > key then
    ...
    depths[0] = key
endif
print(depths[1])
```

write

Question 1

After inserting the first two unsorted items from [5,2,4,3], state the array at each stage. [4] Source: Original OCR J277-style practice question.

Question 2

Explain the purpose of the sorted section in insertion sort. [3] Source: Original OCR J277-style practice question.

Question 3

A sorted prefix is [2,6,8] and next key is 5. State the resulting prefix after insertion. [2] Source: Original OCR J277-style practice question.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Exit Ticket · Answer before checking

Binary search can safely discard half only when data is sorted.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. When the left item is greater than the right item.

Q2. C — Reads the next line

Q3. It holds instructions and working data that the CPU needs while programs run.

Q4. Unsaved work is in volatile RAM, so it is lost when power goes off. Save it to non-volatile secondary storage to keep it.

I Do / We Do pages · Guided practice

Q1. After inserting 2: [2,6,4,1] [2]; after inserting 4: [2,4,6,1] [2].

Q2. Shift 6 right [1], shift 4 right [1]; insert 3 after 1 [1], prefix [1,3,4,6].

Q3. Many keys are near their correct position [1], so fewer shifts/comparisons are needed [1] than when elements are far out of order [1].

You Do pages · Code First

```
array depths = [16,8]
key = depths[1]
if depths[0] > key then
    depths[1] = depths[0]
    depths[0] = key
endif
print(depths[1])
```

Expected result: 16

You Do pages · Independent questions

Q1. Insert 2 → [2,5,4,3] [2]; insert 4 → [2,4,5,3] [2].

Q2. Contains elements already in order [1]; next key is placed at its appropriate position [1] to grow sorted section [1].

Q3. [2,5,6,8] [2].

Exit Ticket · Model answer

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

True

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

2. Write a complete new answer in GREEN PEN. Show working if needed: